

PHASE5

Project Development

Implement Client Script & UI Policy (Incident)

Implementation steps, project usage, performance, and screenshot evidence

Date	30/09/2026
Team ID	6ab4ddb0fc666a751b566e9
Project Name	Implement Client Script & UI Policy(Incident)

Introduction

Project Development is the implementation phase in which the planned ServiceNow configuration is created and tested. It covers the UI Policy, UI Policy Actions, onChange, onSubmit, and onCellEdit Client Scripts, followed by integration and functional testing.

Steps Involved in Project Development

Step 1: Access the ServiceNow Instance

Login with the required configuration permissions, open Incident Management, and identify Impact, Urgency, Assignment Group, Assigned To, and State.

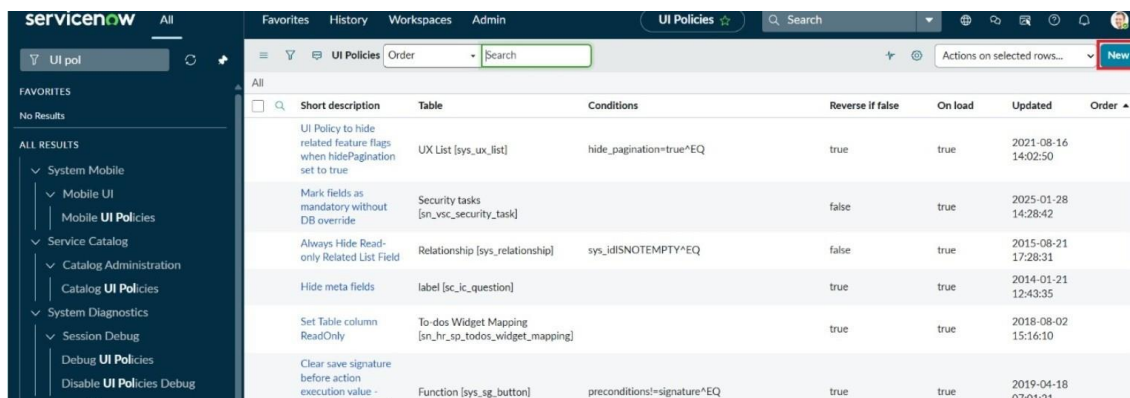


Figure 5.1: ServiceNow development environment.

Step 2: Create the High Impact Control UI Policy

Navigate to System UI -> UI Policies and configure: Name=High Impact Control; Table=Incident; Active=true; Condition = Impact is 1 - High; Reverse if false = true. Configure Assignment Group as mandatory through a related UI Policy Action.

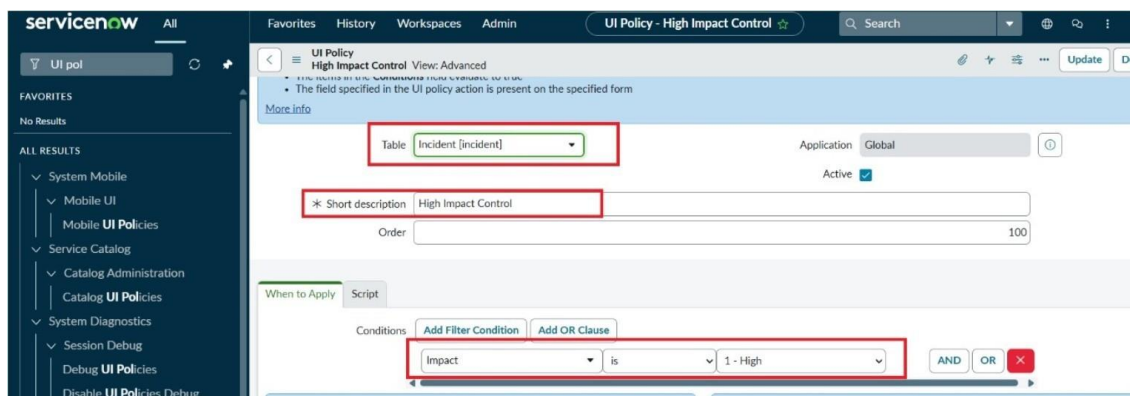


Figure 5.2: High Impact Control UI Policy.

Step 3: Create the Urgency UI Policy Action

Open High Impact Control, create the Urgency action, set Read-only = true, and leave Visible unchanged.

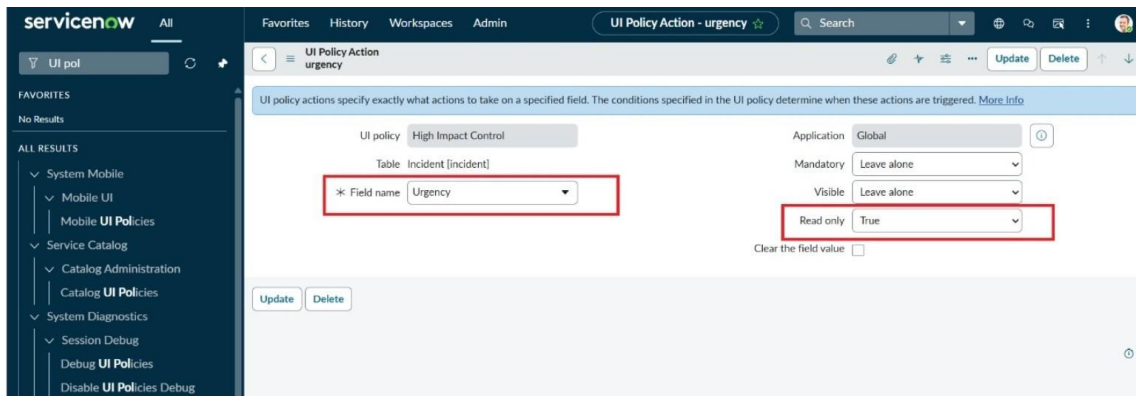


Figure 5.3: Urgency UI Policy Action.

Step 4: Implement the onChange Client Script

Create an onChange Client Script for the Impact field on the Incident table. When new value is 1, the script sets urgency to 1 and displays an informational message.

```
function onChange(control, oldValue, newValue, isLoading) { if (isLoading || newValue == "")
{ return; } if (newValue == "1") { g_form.setValue("urgency", "1");
g_form.addInfoMessage("Urgency set to High for High impact incident."); }}
```

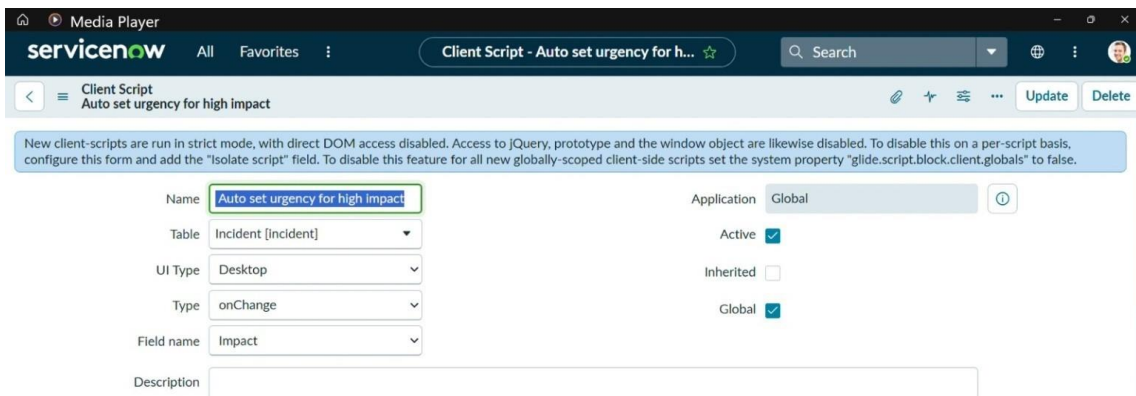


Figure 5.4: onChange Client Script.

Step 5: Implement the onSubmit Client Script

Create an onSubmit Client Script for the Incident table. For High Impact, if Assigned To is empty, show an error and return false to block submission.

```
function onSubmit() { if (g_form.getValue("impact") == "1" && g_form.getValue("assigned_to")
== "") { g_form.showErrorBox("assigned_to", "Assigned To is mandatory for High impact incidents." );
return false; } return true; }
```

Step 6: Implement the onCellEdit Client Script

Create an onCellEdit client script for the State field on the Incident table. The script displays an alert and calls `callback(false)` to block the list edit.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) { alert("State cannot be updated using list editing. Please open the Incident."); callback(false); }
```

The screenshot shows the ServiceNow Client Script configuration interface. The title bar reads 'Client Script - Prevent state change via list edit'. A blue informational banner states: 'New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.' The configuration form includes the following fields: Name (Prevent state change via list edit), Table (Incident [Incident]), UI Type (Desktop), Type (onCellEdit), Field name (State), Application (Global), Active (checked), Inherited (unchecked), and Global (checked). A Description field is also present but empty.

Figure 5.6: onCellEdit Client Script.

Step 7: Integration and Testing

After all components are configured, test them together in the Incident form and Incident list. Verify High Impact behavior, Assigned To validation, reverse conditions, list editing restriction, and form-based State updates.

The screenshot displays the ServiceNow Incident form titled 'Incident - Create INC0010001'. At the top, there are two red error messages: 'The following mandatory fields are not filled in: Short description, Assignment group' and 'The following mandatory fields are not filled in: Short description'. The form contains several input fields and dropdown menus: Number (INC0010001), Caller (Adela Cervantsz), Category (Inquiry / Help), Subcategory (-- None --), Service (empty), Channel (-- None --), State (New), Impact (1 - High), Urgency (1 - High), and Priority (1 - Critical). The 'Submit' and 'Resolve' buttons are visible at the top right.

Figure 5.7: High Impact behavior test.

Use of This Project

The project is intended for Incident Management users such as Service Desk Agents and Incident Management personnel. It can be used to enforce complete Incident information, automate important field values, validate submissions, and control how Incident State is edited.

Application	Use
Incident Creation	Encourages complete and consistent Incident entry.
Incident Prioritization	Automatically sets Urgency to High when Impact is High.
Incident Assignment	Enforces conditional assignment requirements.
Data Validation	Blocks specified incomplete High Impact submissions.
Incident Tracking	Restricts direct State changes through list editing.

The screenshot shows the ServiceNow 'Incident - Create' form. At the top, there are two red error messages: 'The following mandatory fields are not filled in: Short description, Assignment group' and 'The following mandatory fields are not filled in: Short description'. Below these, the form fields are organized into two columns. The left column includes 'Number' (INC0010001), 'Caller' (Adela Cervantsz), 'Category' (Inquiry / Help), 'Subcategory' (-- None --), and 'Service'. The right column includes 'Channel' (-- None --), 'State' (New), 'Impact' (1 - High), 'Urgency' (1 - High), and 'Priority' (1 - Critical). The 'Submit' and 'Resolve' buttons are visible at the top right of the form.

Figure 5.10: Practical use of the developed system.

Performance

Performance is evaluated through functional behavior and repeatability of the configured rules. The supplied project does not provide measured response times or quantitative performance targets, so the results should be recorded after actual testing.

Performance Criterion	Expected Observation
Accuracy	Configured mandatory and validation rules behave consistently.

Performance Criterion	Expected Observation
Responsiveness	Client-side updates and validation occur promptly during user actions.
Reliability	Repeated tests produce the same configured behavior.
Usability	Users receive clear informational or error messages.
Maintainability	Authorized administrators can review and modify the configuration.

The screenshot shows the ServiceNow 'Incident - Create' form. The form is titled 'Incident - Create INC0010001'. It includes a search bar and navigation tabs like 'All', 'Favorites', and 'History'. The form fields are organized into two columns. The left column contains fields for 'Number' (INC0010001), 'Caller' (Adela Cervantsz), 'Category' (Inquiry / Help), 'Subcategory' (-- None --), and 'Service'. The right column contains fields for 'Channel' (-- None --), 'State' (New), 'Impact' (1 - High), 'Urgency' (1 - High), and 'Priority' (1 - Critical). There are two red error messages at the top of the form: 'The following mandatory fields are not filled in: Short description, Assignment group' and 'The following mandatory fields are not filled in: Short description'. The 'Submit' button is highlighted in green.

Figure 5.13: Successful Incident submission.

Testing Result Record

Test Case	Actual Result
Automatic field update	To be recorded after implementation.
Mandatory field enforcement	To be recorded after testing.
Save-time validation	To be recorded after testing.
Reverse condition	To be recorded after testing.
List editing restriction	To be recorded after testing.
Successful Incident submission	To be recorded after testing.

Conclusion

The Project Development phase implements the planned ServiceNow configuration and defines how the project is used and evaluated. The final status of each test should be filled from the screenshots and results observed in the actual ServiceNow instance.